

Viés na Geração de Código Python para a Torre de Hanói com Few-Shot e Chain-of-Thought

Raul Danton Medrado

Bianca Souza Ferreira

Inteligência artificial utilizada: Chat GPT 

Few Shot

Prompt utilizado:

Olá, chat, gere um código de Python para calcular o menor número possível de movimentos para resolver a Torre de Hanói. Exemplos: para N=1, é necessário 1 movimento; para N=2, são necessários 3 movimentos; para N=3, são necessários 7 movimentos e por aí vai. Agora crie um código em Python para qualquer valor de N

Código:

```
E: > python > # Torre de Hanói.py > ...
1  # Torre de Hanói
2  # Calcula o menor número possível de movimentos para qualquer valor de N
3
4  # Fórmula:
5  # movimentos = 2^N - 1
6
7  n = int(input("Digite a quantidade de discos (N): "))
8
9  movimentos = 2 ** n - 1
10
11 print(f"Para N = {n}, o menor número de movimentos é {movimentos}.")
```

Chain of Thought

Prompt utilizado:

A Torre de Hanói segue uma lógica baseada nos recursos, e, para concluí-la fazendo o mínimo possível de jogadas, precisamos pensar: se temos N discos, precisamos mover N-1 discos para outro pino e depois mover outro pino (N-1).

Ou seja, as jogadas (N) = $2 * \text{jogadas (N-1)} + 1$

Dado essa lógica, crie um código em Python para calcular o número de movimentos para N discos

Código:

```
E: > python > # Função para calcular o número mínimo d.py > ...
1  # Função para calcular o número mínimo de movimentos
2  def hanoi(n):
3      if n == 1:
4          return 1
5      return 2 * hanoi(n - 1) + 1
6
7  # Entrada do usuário
8  n = int(input("Digite o número de discos: "))
9
10 # Saída
11 print(f"Para {n} discos, o número mínimo de movimentos é {hanoi(n)}")
```

Análises

Acurácia

Tanto Few Shot quanto Chain of Thought deram resultados corretos: N3 = 7, N4 = 15 e N5 = 31, e conseguem dar resultados iguais mesmo quando saem dos exemplos pedidos: N7 = 127, N9 = 51 e N13 = 8191.

Tempo de resposta

Few Shot: O tempo de resposta foi de aproximadamente 9 segundos (rápido)

Chain of Thought: O tempo de resposta foi de aproximadamente 7 segundos (bem rápido)

O fato da técnica Chain of Thought ter respondido mais rápido me surpreendeu, pois, como Few Shot fica limitado aos exemplos dados, eu imaginei que teria a resposta mais rápida.

Eficácia do código

A técnica Few Shot claramente cria um código mais simples, com apenas 3 linhas de código em si, ele parece ter a mesma eficácia e chance de quebra que o código criado pelo Chain of Thought.

Por mais que Chain of Thought tenha gerado um código com o dobro de linhas de código reais que a de Few Shot, ela ainda faz a exata mesma coisa, porém ela é mais difícil de ser lida por usar mais códigos.

Identificação de Vieses:

Não consegui identificar nenhum viés em nenhum dos códigos gerados, mas, como eu imagino que códigos gerados em few-shot sejam mais diretos, eles têm menos chances de gerarem códigos pensados para evitar bugs.

Conclusões

1 - O mais eficaz?

A técnica de Few Shot se mostrou extremamente efetiva para a resolução dos problemas apresentados, gerou apenas o necessário e funcionou perfeitamente para a situação.

2 - Em quais situações usar cada técnica?

Em casos de problemas simples, como, por exemplo, problemas matemáticos, utilizar a técnica de Few Shot para gerar códigos me parece mais prático.

Enquanto Chain of Thought me parece mais ideal para situações mais complicadas, códigos que necessitam de variáveis, funções e vetores.

3 - Qual foi seu aprendizado?

Eu aprendi que cada técnica funciona melhor dependendo da situação, que não necessariamente Chain of Thought vai gerar respostas melhores e como utilizar as técnicas na prática.